MİGRATİONS

Migrations, Django’nun modellerinizde yaptığınız değişiklikleri (alan ekleme, model silme vb.) veritabanı şemanıza yayma yöntemidir. Çoğunlukla otomatik olacak şekilde tasarlanmışlardır, ancak migrasyonları ne zaman oluşturmanız, ne zaman çalıştırmanız gerektiğini ve karşılaşabileceğiniz yaygın sorunları bilmeniz gerekir.

Komutlar

Veritabanı şemalarının Django tarafından işlenmesi ve migration'larla etkileşim kurmak için kullanacağınız birkaç komut vardır:

Migrate=Göç işlemlerini uygulamaktan ve geri almaktan sorumlu olan birim.

Makemigrations=Bu, modellerinizde yaptığınız değişikliklere göre yeni geçişler oluşturmaktan sorumludur.

Sqlmigrate=Bu, bir geçiş işlemi için SQL ifadelerini görüntüler.

Showmigrations=Bu, bir projenin geçişlerini ve durumlarını listeler.

Veritabanı şemanız için bir sürüm kontrol sistemi olarak geçişleri düşünmelisiniz. `migration`, **makemigrations** model değişikliklerinizi ayrı ayrı geçiş dosyalarına paketlemekten (tıpkı commit'ler gibi) ve **migrate** bu değişiklikleri veritabanınıza uygulamaktan sorumludur.

Her uygulama için geçiş dosyaları, o uygulamanın içindeki "migrations"dizininde bulunur ve kod tabanının bir parçası olarak işlenip dağıtılmak üzere tasarlanmıştır. Bu dosyaları bir kez geliştirme makinenizde oluşturmalı ve ardından aynı geçişleri meslektaşlarınızın makinelerinde, test makinelerinizde ve nihayetinde üretim makinelerinizde çalıştırmalısınız.

Not:
Uygulama bazında, geçişleri içeren paketin adını ayarı değiştirerek geçersiz kılmak mümkündür.(**migration_modules**)

Geçiş işlemleri aynı veri kümesi üzerinde aynı şekilde çalışacak ve tutarlı sonuçlar üretecektir; yani geliştirme ve test ortamlarında gördüğünüz şey, aynı koşullar altında üretim ortamında da aynen gerçekleşecektir.

Django, modellerinizde veya alanlarınızda yapılan her türlü değişiklik için (veritabanını etkilemeyen seçenekler de dahil) geçişler oluşturur; çünkü bir alanı doğru şekilde yeniden oluşturmasının tek yolu, tüm değişikliklerin geçmişte kayıtlı olmasıdır ve bu seçeneklere daha sonra bazı veri geçişlerinde (örneğin, özel doğrulayıcılar ayarladıysanız) ihtiyacınız olabilir.

Arka Uç Desteği

Django'nun birlikte sunduğu tüm arka uçlarda ve şema değişikliğini (SchemaEditor sınıfı aracılığıyla) destekleyecek şekilde programlanmış üçüncü taraf arka uçlarda da migrasyonlar desteklenir .

Ancak, şema geçişleri söz konusu olduğunda bazı veritabanları diğerlerinden daha yeteneklidir; dikkat edilmesi gereken bazı noktalar aşağıda açıklanmıştır.

PostgreSQL

PostgreSQL, şema desteği açısından buradaki tüm veritabanları arasında en yetenekli olanıdır.

MySQL

MySQL, şema değiştirme işlemleri etrafında işlem desteği sağlamaz; bu da bir geçiş işlemi uygulanamazsa, tekrar denemek için değişiklikleri manuel olarak geri almanız gerektiği anlamına gelir (daha önceki bir noktaya geri dönmek mümkün değildir).

MySQL 8.0, DDL işlemleri için önemli performans iyileştirmeleri getirerek , bu işlemleri daha verimli hale getirdi ve tam tablo yeniden oluşturma ihtiyacını azalttı. Ancak, kilitlenmelerin veya kesintilerin tamamen ortadan kalkmasını garanti edemez. Kilitlenmenin hala gerekli olduğu durumlarda, bu işlemlerin süresi ilgili satır sayısıyla orantılı olacaktır.

Son olarak, MySQL'in bir indeksin kapsadığı tüm sütunların toplam boyutu konusunda nispeten küçük bir sınırı vardır. Bu, diğer arka uçlarda mümkün olan indekslerin MySQL altında oluşturulamayacağı anlamına gelir.

SQLite

SQLite'ın yerleşik şema değiştirme desteği çok sınırlıdır, bu nedenle Django bunu şu şekilde taklit etmeye çalışır:

-Yeni şema ile yeni bir tablo oluşturuluyor.

-Verilerin kopyalanması

-Eski masayı devirmek

-Yeni tablonun adını orijinal adla eşleşecek şekilde değiştirme.

Bu işlem genellikle iyi çalışır, ancak yavaş ve zaman zaman hatalı olabilir. Risklerin ve sınırlamaların çok iyi farkında olmadığınız sürece, SQLite'ı üretim ortamında çalıştırmanız ve taşımanız önerilmez; Django'nun sağladığı destek, geliştiricilerin tam bir veritabanına ihtiyaç duymadan daha az karmaşık Django projeleri geliştirmek için yerel makinelerinde SQLite kullanmalarına olanak sağlamak üzere tasarlanmıştır.

İş Akışı

Django sizin için geçiş dosyaları oluşturabilir. Modellerinizde değişiklikler yapın - örneğin, bir alan ekleyin ve bir modeli kaldırın - ve ardından şunu çalıştırın **makemigrations:**

```
$ python manage.py makemigrations

Migrations for 'books':

 books/migrations/0003_auto.py:

 ~ Alter field author on book
```

Modelleriniz taranacak ve mevcut geçiş dosyalarınızdaki sürümlerle karşılaştırıkacak, ardından yeni bir geçiş sati yazılacaktır. Değişikliklerinizin ne olduğunu anlamak için çıktığı okuduğunuzdan emin olun; mükemmel değil ve karmaşık değişiklikler için beklediğiniz sonucu algılamayabilir.

Yeni geçiş dosyalarınızı aldıktan sonra, beklendiği gibi çalıştıklarından emin olmak için bunları veritabanınıza uygulamanız gerekir:(makemigrations)

```
$ python manage.py migrate

Operations to perform:

 Apply all migrations: books

Running migrations:

 Applying books.0003_auto... OK
```

Geçiş işlemi uygulandıktan sonra, geçişi ve model değişikliklerini sürüm kontrol sisteminize tek bir commit olarak kaydedin; bu sayede diğer geliştiriciler (veya üretim sunucularınız) kodu kontrol ettiklerinde, hem modellerinizdeki değişiklikleri hem de eşlik eden geçişi aynı anda alacaklardır.

Eğer geçiş dosyalarına otomatik olarak oluşturulmuş bir ad yerine anlamlı bir ad vermek istiyorsanız, şu seçeneği kullanabilirsiniz:(makemigrations --name)

```
$ python manage.py makemigrations --name changed_my_model
your_app_label
```

Sürüm kontrolü

Geçişler sürüm kontrolünde saklandığı için, bazen siz ve başka bir geliştiricinin aynı anda aynı uygulamaya bir geçiş işlemi uyguladığı ve bunun sonucunda aynı numaraya sahip iki geçiş işlemi oluştuğu durumlarla karşılaşabilirsiniz.

Merak etmeyin, sayılar sadece geliştiricilerin referansı içindir, Django'nun önemsediği tek şey her geçişin farklı bir isme sahip olmasıdır. Geçişler, aynı uygulamadaki önceki geçişler de dahil olmak üzere, hangi diğer geçişlere bağımlı olduklarını dosyada belirtir; bu nedenle, aynı uygulama için sıralanmamış iki yeni geçiş olduğunda bunu tespit etmek mümkündür.

Bu durumda Django size bir uyarı verecek ve bazı seçenekler sunacaktır. Eğer yeterince güvenli olduğunu düşünürse, iki geçiş dosyasını otomatik olarak doğrusal hale getirmeyi teklif edecektir. Aksi takdirde, geçiş dosyalarını kendiniz değiştirmeniz gerekecektir - endişelenmeyin, bu zor değil ve aşağıdaki Geçiş Dosyaları bölümünde daha ayrıntılı olarak açıklanmıştır .

İşlemler

DDL işlemlerini destekleyen veritabanlarında (SQLite ve PostgreSQL), tüm geçiş işlemleri varsayılan olarak tek bir işlem içinde çalışır. Buna karşılık, bir veritabanı DDL işlemlerini desteklemiyorsa (örneğin MySQL, Oracle), tüm işlemler işlem olmadan çalışır.

atomicBir geçiş işleminin bir işlem içinde çalışmasını, ilgili özniteliği olarak ayarlayarak engelleyebilirsiniz **False**. Örneğin:

```
from Django.db import migrations

class Migration(migrations.Migration):

    atomic = False
```

Ayrıca, `transition` kullanarak veya ` transition` parametresini **atomic()** geçirerek geçişin bazı kısımlarını bir işlem içinde yürütmek de mümkündür . Daha fazla bilgi için Atomik Olmayan Geçişler bölümüne bakın . **atomic=TrueRunPython**

Bağımlılıklar

Veritabanı geçişleri uygulama bazında olsa da, modellerinizin ima ettiği tablolar ve ilişkiler, tek bir uygulama için ayrı ayrı oluşturulamayacak kadar karmaşıktır. Başka bir şeyin çalışmasını gerektiren bir geçiş oluşturduğunuzda (örneğin, **ForeignKey** uygulamanıza bir şey eklediğinizde **books**) **authors**, ortaya çıkan geçiş, içindeki bir geçişe bağımlılık içerecektir **authors**.

Bu, geçişleri çalıştırdığınızda, **authors** önce geçişin çalışıp referans verdiği tabloyu oluşturduğu **ForeignKey**, ardından sütunu oluşturan geçişin **ForeignKey** çalışıp kısıtlamayı oluşturduğu anlamına gelir. Eğer bu gerçekleşmeseydi, geçiş, **ForeignKey** referans verdiği tablo mevcut olmadan sütunu oluşturmaya çalışırdı ve veritabanınız bir hata verirdi.

Bu bağımlılık davranışı, tek bir uygulamayla sınırlama getirdiğiniz çoğu geçiş işlemini etkiler. Tek bir uygulamayla sınırlama (ister içinde ister içinde **makemigrations** olsun **migrate**) en iyi çabayı gösterme sözüdür, bir garanti değildir; bağımlılıkların doğru şekilde elde edilmesi için kullanılması gereken diğer uygulamalar kullanılacaktır.

ForeignKey Geçiş (migration) işlemi yapılmamış uygulamaların, geçiş işlemi yapılmış uygulamalarla ilişkisi (, , vb.) olmamalıdır **ManyToManyField**. Bazen çalışabilir, ancak desteklenmemektedir.

Değiştirilebilir bağımlılıklar

django.db.migrations.swappable_dependency (value) ¶
Bu **swappable_dependency()** fonksiyon, uygulama geçişlerinde, değiştirilen modelin uygulamasındaki geçişlere "değiştirilebilir" bağımlılıklar bildirmek için kullanılır; şu anda bu uygulamanın ilk geçişinde kullanılmaktadır. Sonuç olarak, değiştirilen model ilk geçişte oluşturulmalıdır. Argüman, bir uygulama etiketini ve bir model adını tanımlayan **value** bir dizedir ,
örneğin ."**<app label>.<model>""myapp.MyModel"**

`<password>` kullanarak **swappable_dependency()**, geçiş çerçevesine, geçişin değiştirilebilir bir model oluşturan başka bir geçişe bağlı olduğunu bildirirsiniz; bu da gelecekte modelin farklı bir uygulamayla değiştirilmesi olasılığını sağlar. Bu genellikle, Django'nun kimlik doğrulama sistemindeki özel kullanıcı modeli (**settings.AUTH_USER_MODEL** varsayılan olarak `<password> `) gibi özelleştirmeye veya değiştirmeye tabi olan modelleri referans almak için kullanılır."**auth.User"**

Göç dosyaları

Geçişler, burada "geçiş dosyaları" olarak adlandırılan, disk üzerinde bir formatta saklanır. Bu dosyalar aslında, üzerinde anlaşılmış bir nesne düzenine sahip ve bildirimsel bir tarzda yazılmış normal Python dosyalarıdır.

Temel bir geçiş dosyası şu şekilde görünür:

```
from django.db import migrations, models

class Migration(migrations.Migration):

    dependencies = [("migrations", "0001_initial")]

    operations = [

        migrations.DeleteModel("Tribble"),
        migrations.AddField("Author", "rating",

models.IntegerField(default=0)),

    ]
```

Django, bir geçiş dosyasını (Python modülü olarak) yüklediğinde aradığı şey, django.db.migrations.Migration adlı bir alt sınıftır. Daha sonra bu nesneyi dört öznitelik açısından inceler; bunlardan yalnızca ikisi çoğu zaman kullanılır:

-Dependencis= Bu göçün bağlı olduğu göçlerin bir listesi.

-OperationsOperation= Bu geçiş işleminin ne yaptığını tanımlayan sınıfların bir listesi.

Işlemler kilit noktadır;bunlar, Django'ya hangi şema değişiklerinin yapılması gerektiğini söyleyen bir dizi bildirimsel talimattır. Django bunları tarar ve tüm uygulamalara yönelik tüm şema değişikliklerinin bellekte bir temsilini oluşturur ve bunu şema değişikliklerini yapan SQL' i üretmek için kullanır.

Bellekteki bu yapı, modelleriniz ile geçiş dosyalarınızın mevcut durumu arasındaki farkları belirlemek için de kullanılır; Django, en son çalıştırdığınızda modellerinizin durumunu belirlemek için bellekteki bir model kümesi üzerinde tüm değişiklikleri sırayla işler **makemigrations**. Ardından, dosyalarınızdaki modellerle karşılaştırmak için bu modelleri kullanır ve **models.py** neleri değiştirdiğinizi belirler.

Geçiş dosyalarını elle düzenlemenize nadiren, hatta hiç gerek kalmayacaktır, ancak gerekirse bunları manuel olarak yazmanız tamamen mümkündür. Daha karmaşık işlemlerin bazıları otomatik olarak algılanmaz ve yalnızca elle yazılmış bir geçiş dosyası aracılığıyla kullanılabilir, bu nedenle gerekirse bunları düzenlemekten korkmayın.

Özel alanlar

Zaten geçişi yapılmış özel bir alandaki konumlu bağımsız değişken sayısını, bir hata vermeden değiştiremezsiniz **TypeError**. Eski geçiş, değiştirilmiş yöntemi eski imza ile çağıracaktır **__init__**. Bu nedenle, yeni bir bağımsız değişkene ihtiyacınız varsa, lütfen bir anahtar kelime bağımsız değişkeni oluşturun ve yapıcıya şuna benzer bir şey ekleyin.**assert 'argument_name' in kwargs**

Model yöneticiler

İsteğe bağlı olarak yöneticileri migration'lara serileştirebilir ve işlemlerde kullanılabilir hale getirebilirsiniz . Bu , yönetici sınıfında **RunPython** bir öznitelik tanımlayarak yapılır :**use_in_migrations**

```
class MyManager(models.Manager):

    use_in_migrations = True

class MyModel(models.Model):

    objects = MyManager()
```

Eğer bu fonksiyonu dinamik olarak bir yönetici sınıfı oluşturmak için kullanıyorsanız **from_queryset()**, içe aktarılabilir hale getirmek için oluşturulan sınıftan miras almanız gerekir:

```
class MyManager(MyBaseManager.from_queryset(CustomQuerySet)):

    use_in_migrations = True



class MyModel(models.Model):

    objects = MyManager()
```

Göçlerdeki tarihsel modellerle ilgili notlara bakarak ortaya çıkan sonuçları inceleyebilirsiniz.

İlk göçler

`Göç. başlangıç`

Bir uygulamanın "ilk geçişleri", o uygulamanın tablolarının ilk sürümünü oluşturan geçişlerdir. Genellikle bir uygulamanın bir ilk geçişi olur, ancak karmaşık model bağımlılıkları durumunda iki veya daha fazla geçişi olabilir.

İlk geçişler, geçiş sınıfında bir sınıf özniteliği ile işaretlenir . Bir sınıf özniteliği bulunmazsa, bir geçiş, uygulamadaki ilk geçiş ise (yani aynı uygulamadaki başka bir geçişe bağımlılığı yoksa) "ilk" geçiş olarak kabul edilir.**initial = Trueinitial**

Bu seçenek kullanıldığında, bu ilk geçişler özel olarak ele alınır. Bir veya daha fazla tablo oluşturan (işlem) bir ilk geçiş için Django, bu tabloların tümünün veritabanında zaten mevcut olup olmadığını kontrol eder ve mevcutsa geçişi sahte olarak uygular. Benzer şekilde, bir veya daha fazla alan ekleyen (işlem) bir ilk geçiş için Django, ilgili sütunların tümünün veritabanında zaten mevcut olup olmadığını kontrol eder ve mevcutsa geçişi sahte olarak uygular. Bu seçenek kullanılmadığında , ilk geçişler diğer geçişlerden farklı şekilde ele alınmaz.**migrate --fake-initialCreateModelAddField--fake-initial**

Tarihsel tutarlılık

Daha önce de belirtildiği gibi, iki geliştirme dalı birleştirildiğinde geçişleri manuel olarak doğrusallaştırmanız gerekebilir. Geçiş bağımlılıklarını düzenlerken, bir geçişin uygulandığı ancak bazı bağımlılıklarının uygulanmadığı tutarsız bir geçmiş durumu oluşturabilirsiniz. Bu, bağımlılıkların yanlış olduğuna dair güçlü bir göstergedir, bu nedenle Django, düzeltilene kadar geçişleri çalıştırmayı veya yeni geçişler oluşturmayı reddedecektir. Birden fazla veritabanı kullanırken, hangi veritabanlarının tutarlı geçmişi kontrol edeceğini kontrol etmek için veritabanı yönlendiricileri**allow_migrate()** yöntemini kullanabilirsiniz .**makemigrations**

Uygulamalara geçiş dosyaları ekleme

makemigrationsYeni uygulamalar, geçişleri kabul edecek şekilde önceden yapılandırılmış olarak gelir; bu nedenle , bazı değişiklikler yaptıktan sonra bir kez çalıştırarak geçişler ekleyebilirsiniz .

Eğer uygulamanızda zaten modeller ve veritabanı tabloları varsa ve henüz migration'lar yoksa (örneğin, önceki bir Django sürümüne göre oluşturduysanız), migration'ları kullanacak şekilde dönüştürmek için şu komutu çalıştırmanız gerekecektir:

```
$ python manage.py makemigrations your_app_label
```

Bu, uygulamanız için yeni bir başlangıç geçişi oluşturacaktır. Şimdi, komutunu çalıştırın ve Django, bir başlangıç geçişiniz olduğunu *ve* oluşturmak istediği tabloların zaten mevcut olduğunu algılayacak ve geçişi zaten uygulanmış olarak işaretleyecektir. (Bu işaret olmadan, komut hata verecektir çünkü oluşturmak istediği tablolar zaten mevcuttur.) **python manage.py migrate --fake-initialmigrate --fake-initial**. Unutmayın ki bu yöntem yalnızca iki koşul sağlandığında işe yarar:

-Tabloları oluşturduğunuzdan beri modellerinizde herhangi bir değişiklik yapmadınız. Geçişlerin çalışması için *önce* ilk geçişi yapmalı , ardından değişiklikleri yapmalısınız, çünkü Django değişiklikleri veritabanıyla değil, geçiş dosyalarıyla karşılaştırır.

-Veritabanınızı manuel olarak düzenlemediniz - Django, veritabanınızın modellerinizle eşleşmediğini algılayamaz, yalnızca geçişler bu tabloları değiştirmeye çalıştığında hatalar alırsınız.

Göçlerin tersine çevrilmesi

migrateÖnceki geçişin numarasını girerek geçişler geri alınabilir . Örneğin, geçişi geri almak için **books.0003**

```
$ python manage.py migrate books 0002
Operations to perform:
  Target specific migration: 0002_auto, from books
Running migrations:
  Rendering model states... DONE
  Unapplying books.0003_auto... OK
```

Bir uygulama için uygulanan tüm geçişleri geri almak istiyorsanız, şu adı kullanın **zero**:

```
$ python manage.py migrate books zero
Operations to perform:
  Unapply all migrations: books
Running migrations:
  Rendering model states... DONE
  Unapplying books.0002_auto... OK
  Unapplying books.0001_initial... OK
```

Bir geçiş işlemi, geri döndürülemez işlemler içeriyorsa geri döndürülemezdir. Bu tür geçiş işlemlerini geri almaya çalışmak şu hatayı verecektir **IrreversibleError**:

```
$ python manage.py migrate books 0002
Operations to perform:
  Target specific migration: 0002_auto, from books
Running migrations:
   Rendering model states... DONE
  Unapplying books.0003_auto...Traceback (most recent call last):
django.db.migrations.exceptions.IrreversibleError:
 Operation <RunSQL  sql='DROP TABLE demo_books'> in
 books.0003_auto is not reversible
```

Tarihsel modeller

Veritabanı geçişlerini çalıştırdığınızda, Django geçiş dosyalarında saklanan modellerinizin geçmiş sürümlerinden çalışır. Eğer bu işlemi kullanarak Python kodu yazıyorsanız veya veritabanı yönlendiricilerinizde yöntemleriniz **RunPython** varsa , modelleri doğrudan içe aktarmak yerine bu geçmiş model sürümlerini **kullanmanız gerekir .allow_migrate**

Uyarı:
Geçmişe ait modelleri kullanmak yerine modelleri doğrudan içe aktarırsanız, geçişleriniz başlangıçta çalışabilir ancak gelecekte eski geçişleri yeniden çalıştırmaya çalıştığınızda (genellikle yeni bir kurulum yaptığınızda ve veritabanını kurmak için tüm geçişleri çalıştırdığınızda) başarısız olacaktır.
Bu, geçmişe ait model sorunlarının hemen fark edilmeyebileceği anlamına gelir. Bu tür bir hatayla karşılaşırsanız, doğrudan içe aktarmalar yerine geçmişe ait modelleri kullanacak şekilde geçiş dosyasını düzenleyip bu değişiklikleri kaydetmeniz sorun teşkil etmez.

Rastgele Python kodunu serileştirmek imkansız olduğundan, bu geçmişe ait modellerde sizin tanımladığınız özel yöntemler bulunmayacaktır. Bununla birlikte, aynı alanlara, ilişkilere, yöneticilere (yalnızca belirli özelliklere sahip olanlarla sınırlı) ve seçeneklere (ayrıca sürümlü olduklarından mevcut seçeneklerinizden farklı olabilirler) sahip olacaklardır.**use_in_migrations = TrueMeta**

Uyarı:
Bu, geçişlerde nesnelere erişirken özel yöntemlerin çağrılmayacağı **save()** ve özel kurucu veya örnek yöntemlerin bulunmayacağı anlamına gelir. Buna göre plan yapın!

Alan seçeneklerindeki işlevlere yapılan referanslar **upload_to**ve **limit_choices_to** yöneticilerin bulunduğu model yöneticisi bildirimleri geçişlerde serileştirilir, bu nedenle işlevler ve sınıflar, onlara referans veren bir geçiş olduğu sürece saklanmalıdır. Özel model alanları da saklanmalıdır, çünkü bunlar geçişler tarafından doğrudan içe aktarılır.**use_in_migrations = True**
Ek olarak, modelin somut temel sınıfları işaretçiler olarak saklanır, bu nedenle onlara referans içeren bir geçiş (migration) olduğu sürece temel sınıfları her zaman saklamanız gerekir. Olumlu tarafı ise, bu temel sınıflardaki yöntemler ve yöneticiler normal şekilde miras alır, bu nedenle bunlara kesinlikle erişmeniz gerekiyorsa, bunları bir üst sınıfa taşımayı tercih edebilirsiniz.
Eski referansları kaldırmak için, migration dosyalarını sıkıştırabilir veya çok fazla referans yoksa bunları migration dosyalarına kopyalayabilirsiniz.

Model alanlarını kaldırırken dikkate alınması gerekenler

Önceki bölümde açıklanan "geçmişteki işlevlere yapılan atıflar" hususlarına benzer şekilde, özel model alanlarını projenizden veya üçüncü taraf uygulamanızdan kaldırmak, eski geçişlerde bunlara referans verilmesi durumunda sorun yaratacaktır.

Bu durumu kolaylaştırmak için Django, sistem kontrolleri çerçevesini kullanarak model alanlarının kullanım dışı bırakılmasına yardımcı olacak bazı model alanı öznitelikleri sunar .

system_check_deprecated_details Aşağıdaki gibi, özniteliği model alanınıza ekleyin :

```
class IPAddressField(Field):

    system_check_deprecated_details = {

        "msg": (

            "IPAddressField has been deprecated. Support for it
(except "

            "in historical migrations) will be removed in Django
1.9."

        ),

        "hint": "Use GenericIPAddressField instead.",  # optional

        "id": "fields.W900",  # pick a unique ID for your field.

    }
```

İstediğiniz bir süre (Django'nun kendi alanları için iki veya üç özellik sürümü) geçtikten sonra, **system_check_deprecated_details** özniteliği değiştirin **system_check_removed_details** ve sözlüğü aşağıdaki gibi güncelleyin:

```
class IPAddressField(Field):

    system_check_removed_details = {

        "msg": (

            "IPAddressField has been removed except for support in
"

            "historical migrations."

        ),

        "hint": "Use GenericIPAddressField instead.",

        "id": "fields.E900",  # pick a unique ID for your field.

    }
```

__init__() Veritabanı geçişlerinde çalışması için gerekli olan alanın yöntemlerini (örneğin , **deconstruct()**, ve) korumalısınız **get_internal_type()**. Bu alanı, alanı referans alan tüm geçişler var olduğu sürece saklayın. Örneğin, geçişleri birleştirdikten ve eskilerini kaldırdıktan sonra, alanı tamamen kaldırabilirsiniz.

Veri Göçleri

Veritabanı şemasını değiştirmenin yanı sıra, isterseniz şemayla birlikte veritabanındaki verileri değiştirmek için de geçiş işlemlerini kullanabilirsiniz.

Verileri değiştiren geçiş işlemleri genellikle "veri geçişleri" olarak adlandırılır; bunlar şema geçişlerinizin yanında ayrı geçişler olarak yazılmalıdır.

Django, şema geçişlerinde olduğu gibi veri geçişlerini otomatik olarak oluşturamaz, ancak bunları yazmak çok zor değildir. Django'daki geçiş dosyaları İşlemlerden oluşur ve veri geçişleri için kullandığınız ana işlem şudur **RunPython**: .

Öncelikle, üzerinde çalışabileceğiniz boş bir migration dosyası oluşturun (Django dosyayı doğru yere yerleştirecek, bir isim önerecek ve bağımlılıkları sizin için ekleyecektir):

python manage.py makemigrations --empty yourappname

Ardından dosyayı açın; dosya aşağıdakine benzer bir şey olmalı:

```
# Generated by Django A.B on YYYY-MM-DD HH:MMfrom django.db import
migrations

class Migration(migrations.Migration):

    dependencies = [

        ("yourappname", "0001_initial"),

    ]

     operations = []
```

Şimdi yapmanız gereken tek şey yeni bir fonksiyon oluşturmak ve **RunPython** onu kullanmak.
Bu **RunPython** fonksiyon, iki argüman alan çağrılabilir bir nesne bekler; birincisi, geçmişinizdeki geçişin bulunduğu yere uyması için tüm modellerinizin geçmiş sürümlerini içeren bir uygulama kayıt defteridir ve ikincisi , veritabanı şema değişikliklerini manuel olarak gerçekleştirmek için kullanabileceğiniz bir SchemaEditor'dur (ancak dikkat edin, bunu yapmak geçiş otomatik algılayıcısını karıştırabilir).

Şimdi, yeni alanımızı ve **name** değerlerinin birleştirilmiş haliyle dolduracak bir migration yazalım (artık aklımız başımızda ve herkesin ad ve soyadı olmadığını fark ettik). Tek yapmamız gereken, historical modelini kullanmak ve satırlar üzerinde yineleme yapmak:**first_namelast_name**

```
from django.db import migrations

def combine_names(apps, schema_editor):

    # We can't import the Person model directly as it may be a
newer

    # version than this migration expects. We use the historical
version.

    Person = apps.get_model("yourappname", "Person")

    for person in Person.objects.all():

        person.name = f"{person.first_name} {person.last_name}"

        person.save()

class Migration(migrations.Migration):

    dependencies = [

        ("yourappname", "0001_initial"),

    ]

     operations = [

        migrations.RunPython(combine_names),

    ]
```

Bu işlem tamamlandıktan sonra, normal şekilde çalışmaya devam edebiliriz ve veri geçişi diğer geçişlerle birlikte yerinde gerçekleşecektir.**python manage.py migrate**

Geriye doğru geçiş yaparken çalıştırmak istediğiniz mantığı yürütmek için ikinci bir çağrılabilir fonksiyon geçirebilirsiniz **RunPython**. Bu çağrılabilir fonksiyon atlanırsa, geriye doğru geçiş bir istisna hatası verecektir.

Diğer uygulamalardan modellere erişme

Bir fonksiyon yazarken ve **RunPython** bu fonksiyon, migration dosyasının bulunduğu uygulama dışındaki uygulamalardan modeller kullanıyorsa, migration özniteliği ilgili her uygulamanın en son migration dosyasını içermelidir; aksi takdirde, fonksiyon içinde modeli . kullanarak almaya çalıştığınızda **dependencies** şuna benzer bir hata alabilirsiniz:**LookupError: No installed app with label 'myappname'RunPythonapps.get_model()**

Aşağıdaki örnekte, **app1** içinde bulunan modelleri kullanması gereken bir geçiş işlemimiz var **app2**. 'nin ayrıntılarıyla ilgilenmiyoruz, **move_m1** sadece her iki uygulamadaki modellere erişmesi gerekeceği gerçeğiyle ilgileniyoruz. Bu nedenle, 'nin son geçiş işlemini belirten bir bağımlılık ekledik **app2**:

```
class Migration(migrations.Migration):

    dependencies = [

        ("app1", "0001_initial"),

        # added dependency to enable using models from app2 in
move_m1

        ("app2", "0004_foobar"),

    ]

    operations = [

        migrations.RunPython(move_m1),

    ]
```

Daha gelişmiş geçişler

Daha gelişmiş geçiş işlemleriyle ilgileniyorsanız veya kendi geçiş işlemlerinizi yazmak istiyorsanız, geçiş işlemleri referansına ve geçiş yazma kılavuzuna bakın .

Göçleri bastırmak

Veritabanı geçişlerini (migration) özgürce yapmanız ve kaç tane olduğu konusunda endişelenmemeniz teşvik edilir; geçiş kodu, çok fazla yavaşlama olmadan yüzlerce geçişi aynı anda işleyebilecek şekilde optimize edilmiştir. Ancak, sonunda birkaç yüz geçişten sadece birkaçına geri dönmek isteyeceksiniz ve işte burada sıkıştırma (squashing) devreye giriyor.

Sıkıştırarak birleştirme, mevcut birçok göç işlemini aynı değişiklikleri temsil eden tek bir (veya bazen birkaç) göç işlemine indirgeme işlemidir.

Django bunu, mevcut tüm geçiş dosyalarınızı alarak, içlerindeki **Operation** öğeleri çıkarıp sıraya koyarak ve ardından listenin uzunluğunu azaltmaya çalışmak için bunlar üzerinde bir optimizasyon algoritması çalıştırarak yapar; örneğin, ve'nin birbirini götürdüğünü ve'nin **CreateModel** içine dahil edilebileceğini bilir .**DeleteModelAddFieldCreateModel**

İşlem dizisi olabildiğince kısaltıldıktan sonra (mümkün olan miktar, modellerinizin ne kadar iç içe geçmiş olduğuna ve herhangi bir işlem olup olmadığına bağlıdır **RunSQL** (**RunPython** bunlar, olarak işaretlenmedikçe optimize edilemez **elidable**)), Django bunu yeni bir dizi geçiş dosyasına geri yazacaktır.

Bu dosyalar, daha önce sıkıştırılmış olan geçiş dosyalarının yerini alacak şekilde işaretlenmiştir, böylece eski geçiş dosyalarıyla birlikte var olabilirler ve Django, geçmişte nerede olduğunuza bağlı olarak aralarında akıllıca geçiş yapacaktır. Sıkıştırma işleminde henüz ilerlemediyseniz, sona ulaşana kadar bunları kullanmaya devam edecek ve ardından sıkıştırılmış geçmişe geçecektir; yeni kurulumlar ise yeni sıkıştırılmış geçiş dosyasını kullanacak ve eski olanların tümünü atlayacaktır.

Bu sayede, henüz tam olarak güncellenmemiş, üretimde olan sistemleri bozmadan eski dosyaları sıkıştırabilirsiniz. Önerilen süreç, eski dosyaları koruyarak sıkıştırmak, değişiklikleri kaydetmek ve yayınlamak, tüm sistemler yeni sürümle güncellenene kadar beklemek (veya üçüncü taraf bir proje iseniz, kullanıcılarınızın herhangi bir sürümü atlamadan sürümleri sırayla yükseltmelerini sağlamak) ve ardından eski dosyaları kaldırmak, değişiklikleri kaydetmek ve ikinci bir sürüm yayınlamaktır.

Tüm bu işlemleri destekleyen komut şu şekildedir: **squashmigrations** Birleştirmek istediğiniz uygulama etiketini ve geçiş adını ona iletin, gerisini o halleder:

```
$ ./manage.py squashmigrations myapp 0004


Will squash the following migrations: - 0001_initial -
0002_some_change - 0003_another_change - 0004_undo_something


Do you wish to proceed? [y/N] y


Optimizing...  Optimized from 12 operations to 7 operations.


Created new squashed migration
/home/andrew/Programs/DjangoTest/test/migrations/0001_squashed_000
4_undo_something.py  You should commit this migration but leave
the old ones in place;  the new migration will be used for new
installs. Once you are sure  all instances of the codebase have
applied the migrations you squashed,  you can delete them.
```

Sıkıştırılmış geçiş dosyasının adını otomatik olarak oluşturulan bir ad yerine kendiniz belirlemek istiyorsanız bu seçeneği kullanın .**squashmigrations --squashed-name**

Django'da model bağımlılıklarının çok karmaşık olabileceğini ve birleştirmenin (squashing) çalışmama ile sonuçlanabileceğini unutmayın; bu durum ya yanlış optimize edilmiş (bu durumda tekrar deneyebilirsiniz **--no-optimize**, ancak bir sorun da bildirmelisiniz) ya da bir hata (.g.h.) ile **CircularDependencyError** sonuçlanabilir; bu durumda sorunu manuel olarak çözebilirsiniz.

Bir sorunu manuel olarak çözmek için **CircularDependencyError**, döngüsel bağımlılıktaki ForeignKey'lerden birini ayrı bir migration'a ayırın ve diğer uygulamaya olan bağımlılığı da onunla birlikte taşıyın. Emin değilseniz, **makemigrations** modellerinizden tamamen yeni migration'lar oluşturmanız istendiğinde bu sorunu nasıl ele aldığını inceleyin. Django'nun gelecekteki bir sürümünde, **squashmigrations** bu hataları kendi başına çözmeye çalışacak şekilde güncellenecektir.

Geçiş işlemini tamamladıktan sonra, değiştirdiği geçiş işlemleriyle birlikte bu değişikliği kaydetmeli ve uygulamanızın çalışan tüm örneklerine dağıtmalısınız. **Migrate** Değişikliğin veritabanlarında saklanması için bu örneklerin çalıştırıldığından emin olmalısınız.

Ardından, sıkıştırılmış geçiş işlemini aşağıdaki adımları izleyerek normal bir geçiş işlemine dönüştürebilirsiniz:

-Yerine geçtiği tüm geçiş dosyalarını siliyor.

-Silinen geçiş dosyalarına bağlı olan tüm geçiş dosyaları, bunun yerine birleştirilen geçiş dosyasına bağlı olacak şekilde güncelleniyor.

-Sıkıştırılmış geçiş dosyasının sınıfındaki **replaces** özniteliği kaldırmak (Django, bunun sıkıştırılmış bir geçiş dosyası olduğunu bu şekilde anlar).**Migration**

Not

Sıkıştırılmış geçiş dosyalarını normal geçiş dosyalarına dönüştürmeden de sıkıştırabilirsiniz; bu, her ortamın henüz orijinal sıkıştırılmış geçiş kümesini çalıştırmadığı durumlarda yararlı olabilir. Ancak genel olarak, eski geçiş dosyalarını temizleyebilmek için sıkıştırılmış geçiş dosyalarını normal geçiş dosyalarına dönüştürmek daha iyidir.

Silinen geçişlere yapılan referansların budanması

Silinen bir migration dosyasının adını gelecekte tekrar kullanma olasılığınız varsa, bu dosyayı Django'nun migrations tablosundan ilgili seçenekle kaldırmanız gerekir.

Değerlerin serileştirilmesi

Migrasyonlar, modellerinizin eski tanımlarını içeren Python dosyalarıdır; bu nedenle, migrasyon yazmak için Django'nun modellerinizin mevcut durumunu alıp bir dosyaya serileştirmesi gerekir.

Django çoğu şeyi serileştirebilirken, bazı şeyleri geçerli bir Python gösterimine dönüştüremeyiz; bir değerin nasıl koda geri dönüştürüleceğine dair bir Python standardı yoktur (**repr()** yalnızca temel değerler için geçerlidir ve içe aktarma yollarını belirtmez).

Django aşağıdaki verileri serileştirebilir:

-int, **float**, **bool**, **str**, **bytes**, **None**,**NoneType**

-list, **set**, **tuple**, **dict**,**range**

-datetime.date, **datetime.time**, ve **datetime.datetime** örnekleri (saat dilimine duyarlı olanlar dahil)

-**zoneinfo.ZoneInfo** örnekler

-decimal.Decimal örnekler

-**enum.Enum** ve **enum.Flag** örnekler

-uuid.UUID örnekler

-**functools.partial()** ve serileştirilebilir , ve değerlerine **functools.partialmethod**sahip örnekler**funcargskeywords**

-.NET'ten elde edilen saf ve somut yol nesneleri **pathlib**. Somut yollar, saf yol eşdeğerlerine dönüştürülür, örneğin **pathlib.PosixPath**.NET'e. **pathlib.PurePosixPath**

-**os.PathLike** ,örnekler, örneğin **os.DirEntry**, dönüştürülen **str** veya **bytes** kullanılanlar**os.fspath()**

-LazyObject ,Serileştirilebilir bir değeri saran örnekler

-Numaralandırma türleri (örneğin **TextChoices**veya **IntegerChoices**) örnekleri

-Herhangi bir Django alanı

-Herhangi bir fonksiyon veya metot referansı (örneğin **datetime.datetime.today**) (modülün en üst düzey kapsamı içinde olmalıdır)

---İşlevler, uygun şekilde paketlenirse, yani şu yöntem kullanılırsa, süslenebilir: **functools.wraps()**

---Dekoratörler açıkça desteklenmektedir **functools.cache()**.**functools.lru_cache()**

-Sınıf gövdesi içinden kullanılan bağlı olmayan yöntemler

-Herhangi bir sınıf referansı (modülün en üst düzey kapsamı içinde olmalıdır)

-Özel bir **deconstruct()** yöntemi olan her şey (aşağıya bakınız)

Django seri hale getiremiyor:

-İç içe sınıflar

-Rastgele sınıf örnekleri (ör.)**MyClass(4.3, 5.7)**

-Lambdalar

Özel seri numaralandırıcılar

Özel bir seri hale getirici yazarak diğer türleri de seri hale getirebilirsiniz. Örneğin, Django varsayılan olarak seri hale getirme yapmıyorsa **Decimal**, şunu yapabilirsiniz:

```
from decimal import Decimal

from django.db.migrations.serializer import BaseSerializerfrom
django.db.migrations.writer import MigrationWriter

class DecimalSerializer(BaseSerializer):

    def serialize(self):

        return repr(self.value), {"from decimal import Decimal"}

MigrationWriter.register_serializer(Decimal, DecimalSerializer)
```

İlk argüman, **MigrationWriter.register_serializer()** serileştiriciyi kullanması gereken bir tür veya türlerin yinelenebilir bir listesidir.

Serileştiricinizin yöntemi **serialize()**, değerin geçişlerde nasıl görünmesi gerektiğine dair bir dize ve geçişte ihtiyaç duyulan tüm içe aktarmaların bir listesini döndürmelidir.

deconstruct()Bir yöntem ekleme

Django'nun kendi özel sınıf örneklerinizi serileştirmesini sağlamak için sınıfa bir metot verebilirsiniz **deconstruct()**. Bu metot hiçbir argüman almaz ve üç öğeden oluşan bir demet döndürmelidir :**(path, args, kwargs)**

-path=`<class>` ifadesi, sınıfın Python yolunu belirtmeli ve sınıf adı en sonda yer almalıdır (örneğin, `<class> **myapp.custom_things.MyClass**`). Eğer sınıfınız modülün en üst düzeyinde mevcut değilse, serileştirilemez.

-args=Bu, sınıfınızın **__init__**metoduna iletilecek konumlu argümanların bir listesi olmalıdır. Bu listedeki her şeyin kendisi de serileştirilebilir olmalıdır.

-kwargs=Bu, sınıfınızın **__init__**metoduna iletilecek anahtar kelime argümanlarının bir sözlüğü olmalıdır. Her değerin kendisi serileştirilebilir olmalıdır.

Not

Bu dönüş değeri , dört öğeden oluşan bir demet döndüren özel alanlar için **deconstruct()** kullanılan yöntemden farklıdır .

Django, tıpkı Django alanlarına referansları yazdığı gibi, verilen argümanlarla sınıfınızın bir örneğini oluşturarak değeri yazacaktır.

Her çalıştırmada yeni bir geçiş dosyasının oluşturulmasını önlemek için , süslenmiş sınıfa **makemigrations** bir yöntem de eklemelisiniz . Bu fonksiyon, durumlar arasındaki değişiklikleri tespit etmek için Django'nun geçiş çerçevesi tarafından çağrılacaktır.**__eq__()**

Sınıfınızın kurucusuna gönderilen tüm argümanlar kendileri serileştirilebilir olduğu sürece, yöntemi eklemek için **@deconstructible** sınıf dekoratörünü kullanabilirsiniz :**django.utils.deconstructdeconstruct()**

```
from django.utils.deconstruct import deconstructible

@deconstructibleclass MyCustomClass:

    def __init__(self, foo=1):

        self.foo = foo

        ...

    def __eq__(self, other):

        return self.foo == other.foo
```

Bu dekoratör, yapıcıya giden argümanları yakalamak ve korumak için mantık ekler ve ardından deconstruct() çağrıldığında bu argümanları tam olarak döndürür.

Birden fazla Django sürümünü destekleme

Modeller içeren üçüncü taraf bir uygulamanın geliştiricisiyseniz, birden fazla Django sürümünü destekleyen geçiş dosyaları (migrations) göndermeniz gerekebilir. Bu durumda, her zaman **desteklemek istediğiniz en düşük Django sürümüyle** çalışmalısınız .**makemigrations**

Geçiş sistemi, Django'nun geri kalanıyla aynı politikaya göre geriye dönük uyumluluğu koruyacaktır; bu nedenle Django XY'de oluşturulan geçiş dosyaları, Django X.Y+1'de değişmeden çalışmalıdır. Ancak geçiş sistemi ileriye dönük uyumluluk garantisi vermez. Yeni özellikler eklenebilir ve Django'nun daha yeni sürümleriyle oluşturulan geçiş dosyaları eski sürümlerde çalışmayabilir.

1.Soru=Django Migrations (Göçler) tam olarak nedir ve neden veritabanı işlemlerinde doğrudan SQL komutları (CREATE TABLE vb.) yazmak yerine bu sistemi tercih ederiz?

Cevap:

Migrations, Django modellerinde (kod düzeyinde) yapılan değişikliklerin veritabanı şemasına aktarılmasını sağlayan bir **sürüm kontrol sistemidir.** Doğrudan SQL yazmak yerine bu sistemin tercih edilme nedenleri şunlardır:

-Veritabanı Bağımsızlığı: Yazılan kodun PostgreSQL, MySQL veya SQLite fark etmeksizin her veritabanında çalışmasını sağlar.

-Ekip Çalışması: Veritabanı yapısındaki değişiklikler kod dosyalarıyla (Git gibi) paylaşılabildiği için ekipteki herkesin veritabanı aynı kalır.

-Geri Alınabilirlik: Yapılan bir hata durumunda veritabanını kolayca eski bir sürüme (versiyona) döndürebilme imkanı tanır.

2.Soru: Django'da yaptığınız model değişikliklerini paketleyip bir "göç dosyası" haline getiren ancak bu değişiklikleri henüz veritabanına **uygulamayan** komut aşağıdakilerden hangisidir?

A) `python manage.py migrate`

B) `python manage.py makemigrations`

C) `python manage.py showmigrations`

D) `python manage.py sqlmigrate`

Cevap:

B) python manage.py makemigrations

Açıklama: makemigrations komutu modellerdeki değişiklikleri algılar ve migrations/ klasörü altında yeni bir dosya oluşturur. Bu değişikliklerin veritabanında fiziksel olarak gerçekleşmesi için ise migrate (A şıkkı) komutunun çalıştırılması gerekir.